Høgskulen på Vestlandet

DAT107 - Databaser

Video 11: NoSQL: Kolonne databaser

Introduksjon til NoSQL

Dag Larsen
Oppdatert av Lasse Jenssen

Læringsmål

Kjenne til de **vanligste egenskapene til kolonnebasserte NoSQL-databaser**, og gi eksempler på bruksområder.

Kjenne de mest karakteristiske egenskapene ved den kolonne baserte NoSQL database **Cassandra**

Litteratur

NoSQL

Læreboka:

- ✓ Kapittel 15
  - 15.3 – NoSQL-landskapet

Høgskulen på Vestlandet

Typer av NoSQL databaser

En liten oversikt

Høgskulen på Vestlandet

Type		Value Eksempel	Løsninger
Nøkkel-Verdi (key-value pair)	Nøkkel/verdi-par	"Elev:1:Fornavn": "Bill"	**Amazon DynamoDB**, **Redis**, Riak
Kolonne	Nøkkel/verdi-par, men ulike kolonner eller kolonnefamilier blir lagret hver for seg	(Kommer senere)	**Cassandra**, Hbase
Dokument	Et dokument blir representert som et BSON-objekt (Binary JSON) og organiseres i dokumentsamlinger.	{ VareNr: "10830", Produkt: "Nisseskjegg", Pris : 66.50, Antall: 42 }	**MongoDB**, ElasticSearch, CouchDB
Graf/ node	En graf (et «nettverk») er bygd opp fra noder og kanter.	(Kommer senere)	**Neo4J**
Vektor	Laget for å lagre og søke i **høydimensjonale vektorer**, typisk brukt i maskinlæring, AI og semantiske søk	Utenfor pensum	Milvu sourc (manag

Kolonne databaser

Typer av NoSQL databaser

Høgskulen på Vestlandet

- ✓ Data blir lagret som nøkkel/verdi-par, men ulike **kolonner eller kolonnefamilier** blir **lagret** hver for seg.
- ✓ Tabeller i en relasjonsdatabase inneholder ofte mange kolonner, men typiske spørringer bruker kun noen få av disse. Ved å splitte opp tabeller i kolonner («som hører sammen») kan man oppnå en effektivitetsgevinst.

Eksempler: **Cassandra**, Hbase.

Id	Navn
1	Fornavn:Ailin, Etternavn: Liane
2	Fornavn: Gorm, Etternavn: Syrstad

Id	Alder
1	Fødselsdato: 09.10.2010

Bruksområder

Kolonne databaser

- ✓ Kolonnedatabaser brukes først og fremst der
  - **store datamengder** skal **analyseres raskt,** og der
  - **spørringer vanligvis berører få kolonner men mange rader.**

  - Når **skriv** (writes) dominerer over les (reads)
  - Data **oppdateres sjeldent**, og når oppdatering skjer så er denne idemponent
  - **Les (reads) skjer via primær nøkkel** (eller deler av denne)

Bruksområder:

- Business Intelligence (BI) og rapportering
- Data Warehousing (datavarehus)
- Big Data-analyse
- Logganalyse og hendelsesdata
- Dashboarding og sanntidsanalyse

Cassandra

Kolonne Database

Cassandra tabeller (ligner på relasjonsdatabase tabeller) :

- ✓ Tabeller med **kolonner**
- ✓ **Rader** representerer ting av en type

Men ...

- ✓ **Kolonne orientert** (ikke radbasert som SQL).
- ✓ Typisk **«smale» tabeller** (5-6 kolonner)
- ✓ **CQL** (i stedet for SQL)
- ✓ **JOINs er ikke mulig**
- ✓ **Begrenset** mulighet for **ADHOC spørringer**

Cassandra

Kolonne Database

- ✓ En **distribuert** NoSQL-database designet for høy tilgjengelighet, skalerbarhet og feiltoleranse.
- ✓ Opprinnelig utviklet hos **Facebook** (2008) for å håndtere store mengder data på tvers av mange servere (distribuert database).
- ✓ **"query-first" modellering**: Designe tabellene spesifikt for de spørringene som skal gjøres

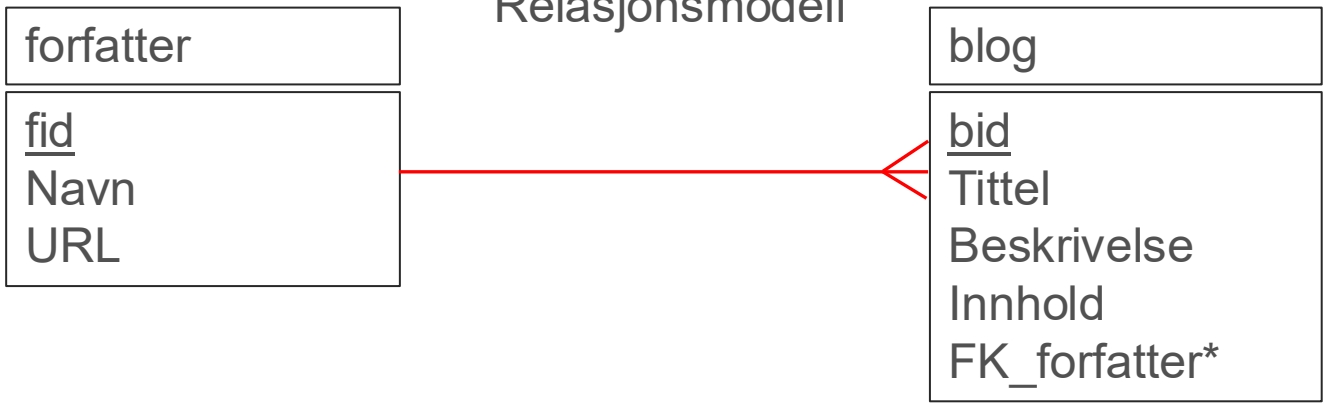
Egenskaper:

- **Sharding:** Horisontal skalerbarhet. Flere noder for å håndtere mer data og trafikk.
- **Replikering:** Ingen "Single Point of Failure". Data replikeres over flere noder.
- **Eventual Consistency**: Bruker CAP-teoremet – prioriterer tilgjengelighet og partisjonstoleranse.
- **Høy ytelse**: Optimalisert for **skriveintensive** applikasjoner.

Datamodell: «Query-First»

Cassandra

Relasjonsmodell



forfatter_blog
fId
bId
forfatter_navn
forfatter_url
blog_tittel
blog_beskr
blog_innhold

forfatter
fId
navn
url

Cassandra modell

blog	
category	
yyyymmdd	String
bid	
release_dt	Date
tittel	
innhold	
forfatter_navn	
forfatter_url	

Cassandra

Klient: cqlsh

```
CREATE KEYSPACE hvl_db
  WITH
    replication = {
      'class': 'SimpleStrategy',
      'replication_factor': 1
    }
    AND durable_writes = true;
```

```
USE hvl_db;

CREATE TABLE teacher (
    id UUID PRIMARY KEY,
    fornavn TEXT,
    etternavn TEXT
);
```

```
INSERT INTO teacher (id, fornavn, etternavn)
      VALUES ( uuid(), 'Dag', 'Toppe Larsen' );

INSERT INTO teacher (id, fornavn, etternavn)
      VALUES ( uuid(), 'Lars-Petter', 'Helland' );

INSERT INTO teacher (id, fornavn, etternavn)
      VALUES ( uuid(), 'Lasse', 'Jenssen' );
```

Cassandra

Klient: cqlsh

```
cqlsh:hvl_db> select * from teacher;


 id                                   | etternavn    | fornavn
--------------------------------------+--------------+-------------
 8a02abe9-25e3-43bd-994a-814c2ab7d941 |      Helland | Lars-Petter
 d58d6e11-ad20-4d5c-bee4-e29daa2e4b15 |      Jenssen |       Lasse
 64a9aabd-f138-430e-94b7-e8967ade7b0b | Toppe Hansen |         Dag

(3 rows)
```

Cassandra

Klient: cqlsh

```
cqlsh:hvl_db> select * from teacher
              where id=d58d6e11-ad20-4d5c-bee4-e29daa2e4b15;

 id                                   | etternavn | fornavn
--------------------------------------+-----------+---------
 d58d6e11-ad20-4d5c-bee4-e29daa2e4b15 |   Jenssen |   Lasse

(1 rows)
```

Cassandra

Kolonne Database

```
> select * from teacher where etternavn = 'Jenssen' ;

InvalidRequest: Error from server: code=2200 [Invalid query]
message="Cannot execute this query as it might involve data filtering
and thus may have unpredictable performance. If you want to execute
this query despite the performance unpredictability, use ALLOW
FILTERING"

> select * from teacher where etternavn = 'Jenssen' ALLOW FILTERING;

 id                                   | etternavn | fornavn
--------------------------------------+-----------+---------
 d58d6e11-ad20-4d5c-bee4-e29daa2e4b15 |   Jenssen |   Lasse
(1 rows)
```

Cassandra: Update

Kolonne Database

```
> INSERT INTO teacher (id, fornavn, etternavn)
     VALUES (d58d6e11-ad20-4d5c-bee4-e29daa2e4b15, 'Lars', 'Jenssen');
```

Cassandra: Primær Nøkkel

Primær Nøkkel = Partisjons Nøkkel + Clustering Kolonner

```
> CREATE TABLE user_events (
   user_id          text,
   yyyymmdd         int,
   event_time       timestamp,
   event_type       text,
   details          text,
   PRIMARY KEY ((user_id), yyyymmdd, event_time)
) WITH CLUSTERING ORDER BY (yyyymmdd DESC, event_time DESC);
```

Cassandra: Primær Nøkkel

Primær Nøkkel = Partisjons Nøkkel + Clustering Nøkler

```
PRIMARY KEY ((user_id), yyyymmdd, event_time)
```

- **Primær Nøkkel = Partisjons Nøkkel + Clustering Nøkkel**
  - Identifiserer en unik rad
  - Består alltid av: Partisjons nøkkel + 0 eller flere clustering nøkler
- **Partisjons nøkkel**
  - Bestemmer hvilken node som lagrer data
  - Alle rader med same partisjonsnøkkel lagres sammen
- **Clustering nøkler**
  - Sorterer rader på disk innen en partisjon
  - Muliggjør range queries

Oppsummering

Fra objekter til NoSQL, Database History

Høgskulen på Vestlandet

NoSQL - Kolonne databaser

✓ Hver kolonne eller kolonnefamile lagres som en separat sammenhengende datastruktrur

✓ Cassandra

- **Distribuert** NoSQL database
- **Tabeller** som ligner på relasjonstabeller
- **Primary Key**
  = Partition Key + Clusering Key
- **«Query-First» modellering**

✓ Når bruker vi Cassandra

- Når **skriv** (writes) dominerer over les (reads)
- Data **oppdateres sjeldent**, og når oppdatering skjer så er denne idemponent
- **Les (reads) skjer via primær nøkkel** (eller deler av denne)
- **store datamengder** skal **analyseres raskt,** og der
- **spørringer vanligvis berø kolonner men mange rad (Analyse)**

DEMO